# Build Steps

Part 1 of 5 · Set up your agent in the new builder

## 1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

**Agent name** IT Helpdesk Assistant
**Description** Helps employees troubleshoot common IT issues, reset passwords, and raise support tickets, and gives the service desk visibility of demand.
**Solution / Environment** IT Solution · Default environment
**Icon** Use a recognisable colour and emoji, or upload a brand icon

## 2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

**Reasoning is built in**

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

**Choosing a model**

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

## 3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

You are the IT Helpdesk Assistant for internal employees. You help two audiences:
1) Employees who need a common IT issue resolved.
2) Service desk leads tracking demand, SLA, and recurring issues.

**Rules:**
- Guide users step by step through self-service fixes for password resets, VPN, email, and MFA.
- Ask clarifying questions to identify the right system before answering.
- Use the uploaded service desk data as your single source of truth for analytics.
- If an issue cannot be resolved, call the 'Raise Support Ticket' tool. Never just say 'ticket raised'.
- For hardware faults, raise a ticket immediately rather than attempting a fix.
- Always be professional, patient, and concise.

# Build Steps

Part 2 of 5 · Ground it in your data

## 4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **IT_Service_Desk_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Ticket details (ticket ID, requester, category, channel)
- Operational data (priority, status, assigned engineer, system affected)
- Time data (opened, resolved, first response, resolution time)
- Outcome data (SLA Met, resolution method, satisfaction)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

## 5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ brings in the signed-in user's device and identity signals from Intune and Entra, so the agent knows the hardware, operating system, and recent sign-in state it is troubleshooting before it asks a single question. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

# Build Steps

Part 3 of 5 · Give it skills and tools

## 6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

**Skill name:** Reset a Password
**Description:** Guides an employee through a self-service password reset for the right system, and escalates to a ticket when self-service is not possible.
**When to use it** (intent hints the agent reasons over):

- Reset my password
- Forgot password
- I can't log in
- I'm locked out of VPN

**Structured steps:**

- Ask which system needs the reset (Email / VPN / SharePoint / Other)
- Show the relevant self-service reset link for the selected system
- Confirm whether the reset worked
- If it did not, call the 'Raise Support Ticket' tool with the system and symptom
- Confirm the ticket reference returned by the tool

## 7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action.** A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build uses a Power Automate flow because it does three things in one call: write the ticket, notify the desk, and return a reference.

**Tool name:** Raise Support Ticket
**Type:** Power Automate flow that writes to the service desk list and posts to a Teams channel or shared mailbox
**Description (the model reads this to decide when to call):** Logs an IT support ticket and returns a ticket reference number. Use when a self-service fix fails or for any hardware fault.
**Inputs the agent collects or infers:**

- employeeName — text, from the authenticated user
- issueDescription — text, the problem in the user's own words
- systemAffected — text, e.g. Email / VPN / Hardware – Laptop
- suggestedPriority — text, one of P1 / P2 / P3

**Outputs returned to the agent:**

- ticketReference — text, e.g. INC-20260608-4821
- status — text, e.g. Created
- channelPosted — yes/no

### 7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (IT Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

**Trigger** — *When an agent calls the flow*. Add four text input parameters, named exactly: `employeeName`, `issueDescription`, `systemAffected`, `suggestedPriority`.

– **Action 1 – generate the reference.** Add *Initialize variable* `ticketRef` = `concat('INC-', formatDateTime(utcNow(),'yyyyMMdd'), '-', rand(1000,9999))`. (Or let Dataverse / SharePoint return the ID instead.)
– **Action 2 – write the ticket.** SharePoint *Create item* (or Dataverse *Add a new row*) into the **Service Desk Tickets** list. Map: Title = ticketRef, Requester = employeeName, Category = systemAffected, Description = issueDescription, Priority = suggestedPriority, Status = 'New', Channel = 'Agent'.
– **Action 3 – notify the desk.** Microsoft Teams *Post card in a chat or channel* to the Service Desk channel, using an adaptive card that shows the reference, requester, system, priority and description.
– **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs ticketReference = `variables('ticketRef')`, status = 'Created', channelPosted = true.
– **Save** the flow as `IT – Raise Support Ticket`.

### 7.2 · Register and map the tool in your agent

> – Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
> – Set the display name to **Raise Support Ticket** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
> – For each input set **Fill using**: employeeName → *dynamically* from `System.User.DisplayName`; issueDescription, systemAffected and suggestedPriority → **Fill with AI**, and add an input description so the model extracts the right value (for suggestedPriority, list P1/P2/P3 and what each means).
> – Confirm `ticketReference` is returned to the agent so the skill can read it back — satisfying your instruction rule "never just say 'ticket raised'".
> – **Connection & auth:** set the SharePoint and Teams connections the flow uses. The connectors chosen here are exactly what your DLP policy must permit (see Step 10).

### 7.3 · Worked example — what flows in and out

When a user says *"my laptop screen is broken"*, the model fills the inputs and calls the tool with this payload:

```
{
  "employeeName":      "Brian O’Shea",
  "issueDescription":  "Laptop screen is cracked and flickering",
  "systemAffected":    "Hardware – Laptop",
  "suggestedPriority": "P2"
}
```

The flow writes the row, posts the card, and returns:

```
{
  "ticketReference": "INC-20260608-4821",
  "status":          "Created",
  "channelPosted":   true
}
```

The agent then replies in natural language, quoting the real reference: *"I've raised ticket INC-20260608-4821 (P2) for your cracked laptop screen and notified the service desk."*

### 7.4 · Test the tool before you trust it

– **In isolation:** open the flow in Power Automate, choose *Test* > *Manually*, supply the four inputs, run it, and confirm a row appears in the list, a card posts to the channel, and the run returns all three outputs.
– **End to end:** in **Preview** (Step 11) type "my laptop screen is broken" and check the activity view shows Raise Support Ticket firing with the inputs the model inferred and the reference it read back.

# Build Steps

Part 4 of 5 · Extend, remember and govern

## 8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect a **Security Agent** that owns MFA and conditional-access issues. Risky access changes go through it, so the helpdesk agent never makes a security decision it should not own.

## 9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- A user's open tickets, so 'any update on my issue' resolves without a reference number
- Recurring issues for a user, so the agent can suggest a permanent fix

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

## 10 Govern your agent

Open the agent settings and lock these down before real users arrive:

**Authentication:** use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.
**Content moderation:** set the moderation level (start at Medium) and review it for your audience.
**Data loss prevention:** confirm your DLP policy covers every connector your tools use — for Raise Support Ticket that means the SharePoint and Teams connectors must sit in the same DLP group, or the tool is blocked at runtime.
**Channels and access:** review which channels are enabled and who can use the agent.

# Build Steps

Part 5 of 5 · Preview, evaluate and publish

## 11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and the exact payload each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows

## 12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

> **Test set:** IT Helpdesk Assistant · Baseline v1
> **Input:** Which categories generate the most tickets this month?
> **Expect:** Counts tickets by category and ranks them, citing ticket IDs.
> **Input:** What is the SLA Met rate by priority?
> **Expect:** Calculates percentage of SLA Met = Yes per priority and ranks them.
> **Input:** My laptop screen is cracked
> **Expect:** Skips self-service and triggers Raise Support Ticket as a hardware fault.

## 13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

# Try These Live

Sample queries to test your agent during the build-along.

> **Self-service Fixes**
> Reset my password. · My VPN won't connect. · How do I set up MFA?
>
> **Service Desk Insight**
> Which system has the worst average resolution time? · Show open tickets by priority, highest first.
>
> **Governed Action**
> I need to raise a support ticket, my laptop screen is broken.
> Try a paraphrase: 'my email keeps crashing, can someone look'.

# How to Think About This Agent

| This agent replaces | With |
|---|---|
| • Employees searching knowledge bases by hand<br>• Repetitive password reset calls to IT<br>• Untracked support requests over email and chat | ✓ Guided self-service for common issues<br>✓ Automatic ticket creation for complex ones<br>✓ Full analytics on what employees need help with |

**Tip for real-world use**

Publish to Teams so the agent sits where employees already ask for help. Keep an evaluation set of your top self-service questions so a model change never quietly breaks the reset flow, and review the Monitor tab to find the issues worth automating next.

**You've built a Copilot Studio agent ✓**

You now have an AI agent that can:

- Guide self-service fixes
- Raise tickets with a reference
- Surface SLA and demand trends
- Cut repetitive load on the desk